DAFS / PROJECT DIFFERENCE

DAFS を長鎖 RNA へ持っていくための 4 つの変更

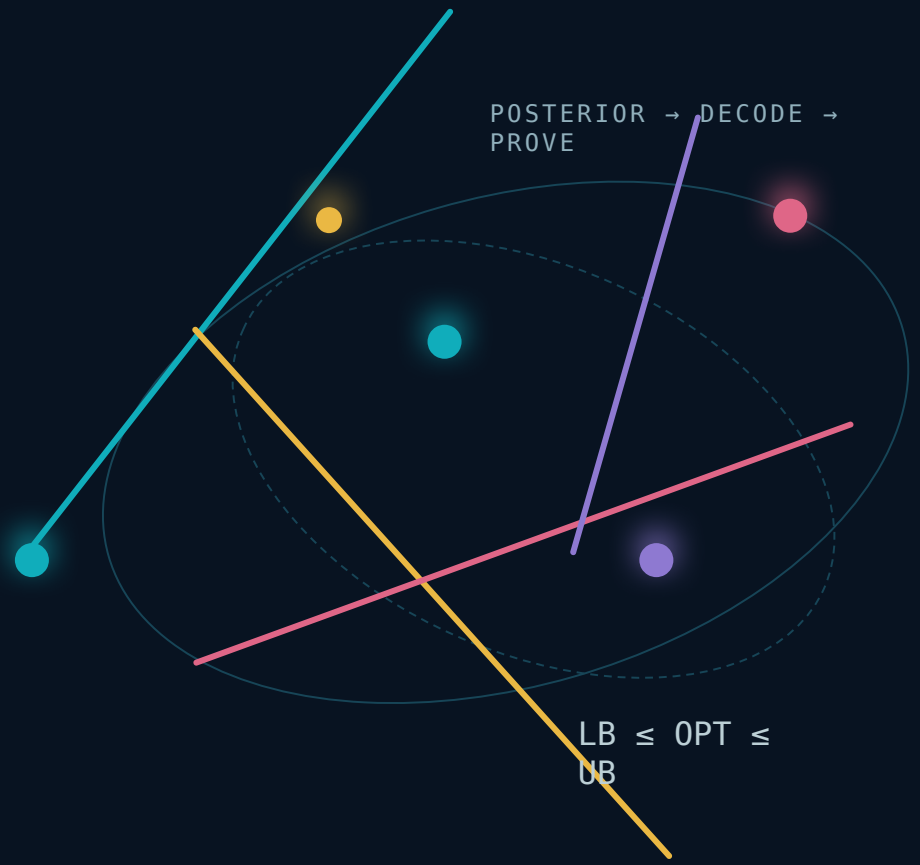
元論文の双対分解を残したまま、長さ依存のボトルネックを線形化し、RIBOSUM ・厳密な UB/LB ・ dynamic CBP を同じ最適化ループへ接続した。

① Linear path

② RIBOSUM

③ certified UB / LB

④ dynamic CBP



基準コミット 9360ad8 · **データ** Murlet / 16S / 23S · **集計** 2026-08-31

Project-local summary · numerical values are stored benchmark means; clean-release revalidation remains future work.

ポイントは「４つの置換」が一本のパイプラインになったこと

速度だけを上げたのではなく、確率・候補・復号・証明を同じ疎な support 上でつないだ。

01

Linear 化

LinearPartition / LinearAlign 、固定 beam 、閾値疎行列、 LinearNussinov / LinearNeedlemanWunsch 。

02

RIBOSUM

pair–pair match に RIBOSUM85-60 を導入。構造・整列・ CBP の joint objective を強化。

03

UB / LB + 修飾 subgradient

beam を UB と誤解せず、 certificate UB と repair LB を分離。 Polyak step を track ごとに整合。

04

dynamic CBP

CBP を全列挙せず、 ADD ・ PRICE ・ PRUNE で working set を更新。 positive reduced cost を逃さない。

長さ方向
固定閾値・固定 beam ・固定反復なら O(n)

目的関数
RIBOSUM を DD / repair / IP で共有

証明
beam search と厳密 relaxation を別トラック

候補
dynamic pricing で CBP を必要時だけ追加

Difference from original DAFS: the dual decomposition is preserved; the probability engines, decoders, CBP management, bounds and update tracks are replaced or extended.

Linear 化：密な計算を、閾値疎行列と固定 beam へ置き換えた

元論文の x / y / z / w の分解は維持し、長さに対して高次になる部分を各段で疎・固定幅にした。

ORIGINAL / DENSE

posterior → dense DP → full candidate

全 posterior 、 Nussinov / Needleman–Wunsch 、 CBP 候補の直積を扱うため、長鎖では計算量とメモリが急増。

THIS PROJECT / LINEAR PATH

LinearPartition / LinearAlign → sparse → fixed beam

閾値以上の candidate だけを保存し、確率計算・ max-sum 復号・ q 更新・最終 folding を固定 beam で処理する。

poster ior

LinearPartition + LinearAlign

固定 beam で BPP / match posterior を近似。

→

sparse support

threshold + sparse q

非零候補と乗数を同じ座標集合で管理。

→

deco der

fixed-beam max-sum

DD の folding / alignment を linear decoder で復号。

BPP beam	alignment beam	threshold	complexity
default 100	default 100	support を固定	固定条件で O(n)

読み方： 「線形」は beam 幅・閾値・反復数・ profile 配列数を固定したときの、 profile 長に対するスケーリング。

Implementation: src/linearalign.cpp · src/linearnussinov.cpp · src/linearneedleman_wunsch.cpp · src/sparse_matrix.h · docs/linearization_paper_draft_ja.md

RIBOSUM： w を「制約補助変数」から「生物学的 score を持つ変数」へ

単独 base-pair probability では区別しにくい、 pair–pair の保存・ compensatory substitution を目的関数へ入れた。

元論文の係数

```
cijkl = ... + 0 · wijkl
```

pair–pair match そのものには、実質的な score を持たせていなかった。

今回の実装

```
cijkl = ... + ρ Rijkl
```

RIBOSUM85-60 を soft objective として付加。負 score を hard filter にはしない。

DD reduced cost

candidate pricing と同じ R を使う

repair LB

実行可能解の score にも入る

IP objective

dual / primal の定義を一致

effect

構造的な pair 型を評価

MURLET / SPS

.8220 → .8222

LinearAlign, ρ=0 → .3

MCC

.6502 → .6772

joint objective の改善

CBP-F1

.5690 → .5898

CBP 整合性も改善

DECK OPERATING POINT

ρ=.3

LinearTurboFold 由来 score

要点： RIBOSUM は別処理ではなく、 DD ・ dynamic CBP ・ repair ・ IP を貫く同じ目的関数の追加項。

RIBOSUM85-60 · LinearAlign は LinearTurboFold 由来の BeamAlign/HMM score を使用 · 本資料の linear 条件は LinearAlign + lpc + ρ=.3 。

UB / LB： 最適値を「下から」と「上から」挟む

DAFS は最大化問題。実行可能解の score は LB 、制約を緩めた問題の厳密解は UB になる。

1 / 下から押さえる

beam → repair

beam で構造・ alignment の候補を探す。 3 者が噛み合わなければ、 intersection / consensus repair で整合させる。

実行可能解の score

score ≤ OPT ⇒ LB

良い解が見つかるほど LB は上がる ↑

対象の DAFS 問題

本当に知りたい値

OPT

全制約を満たす構造・ alignment ・ CBP の最大 score 。直接求めるのが難しいので、両側から挟む。

2 / 上から押さえる

relaxation → certificate

制約を外して候補集合を広げた問題を、 folding certificate や row relaxation で厳密に解く。

緩和問題の厳密な最大値

OPT ≤ relaxed optimum ⇒ UB

証明が強くなるほど UB は下がる ↓

LBt ≤ OPT ≤ UBt この範囲は常に正しい

重要： beam の値をそのまま UB とは呼ばない。 beam 外に、もっと良い解が残っているかもしれないから。 beam は LB を探す役割。

gap = UB − LB
gap が 0 に近いほど、最適値を狭く特定できる。

Polyak step $\eta_t = \alpha \cdot \max(0, D(q_t) - LB_t) / ||g_t||^2$ relaxation の目的値と、その同じ問題の subgradient を対応させる

src/relaxed_bounds.h · src/dafs.cpp:1367–1380, 1630–1645 · docs/linearization_paper_draft_ja.md § 3.7

dynamic CBP： 必要な列だけを生成し、 pricing で漏れを探す

CBP 候補を最初から全列挙せず、現在の解と reduced cost を見ながら working set を更新する。

STEP 0

ADD

現在の x / y / z から、整合しそうな CBP を双方向 separation で候補化。

STEP 0

PRICE

未追加の候補を reduced cost で走査。RIBOSUM 項もここに入る。

STEP 0

PRUNE

stale な列を working set から外し、次の反復を小さく保つ。

STEP 0

RE-SOLVE

DD decoder ・ LB repair ・ UB certificate を同じ candidate 集合で更新。

RESUL

full problem へ近づく

positive reduced-cost column を逃さず、固定 support 上で列生成を続ける。

STATIC / PRE-ENUMERATION

CBP を先に全部作る

候補数が profile 長に伴って膨らみ、 RIBOSUM を入れるほど管理コストも重くなる。

DYNAMIC / WORKING SET

解きながら追加・削除

候補生成と pricing を分け、 score ・ repair ・証明を同じ列集合に通す。 linear path のための実装上の要所。

candidate cache

endpoint 別に再利用

positive score

RIBOSUM positive column を守る

stale pruning

working set を compact に保つ

same objective

DD / LB / IP で一貫

src/dafs.cpp:426–464, 1272–1295, 1650–1660 · src/ribosum.cpp · docs/linearization_paper_draft_ja.md §3.9–3.10

HOW THE FOUR CHANGES CONNECT

元論文との差分を一枚で見ると、こうなる

posterior の近似を入り口に、候補生成・最適化・証明を一本の linearizable loop として実装した。

01
Linear posterior
固定 beam で BPP / match を計算。

→

02
RIBOSUM score
pair–pair match を目的関数へ。

→

04
dynamic CBP
ADD / PRICE / PRUNE 。

→

01
linear DD decoder
fixed-beam max-sum で復号。

→

03
repair LB + certified UB
Polyak track を整合。

ORIGINAL DAFS

decompose

MEA/IP を x / y / z / w へ分解し、 subgradient で整合性を回復する。

THIS PROJECT

decompose + scale + certify

分解の意味を保ちつつ、長さ依存の各部を線形化し、 RIBOSUM ・ dynamic CBP ・厳密 UB/LB を追加。

一言で： 「 DAFS を別の方法にした」のではなく、 DAFS の dual decomposition を長鎖向けの疎・固定幅・証明付き実装へ拡張した。

Murlet： RIBOSUM の重みは nonlinear / linear で分ける

RIBOSUM なし／ありを同じ評価軸で並べ、 linear では ρ=.3 、 nonlinear では ρ=.075 が最も良い条件として比較する。

比較条件

```
nonlinear: CONTRAlign + CONTRAfold  ·  linear: LinearAlign [LinearTurboFold score] + lpc
--no-alifold  ·  --dynamic-cbp  ·  --max-iter=20  ·  OMP_NUM_THREADS=1
```

DATA

Murlet / 13

reference-based accuracy + SCI + resource usage

構成	RIBOSUM P	SPS	MCC	CBP F1	SCI	WALL S	RSS MIB
nonlinear	なし / 0	.8191	.6784	.5919	.6326	.94	11.5
nonlinear	あり / .075	.8224	.6861	.5974	.6590	.94	11.6
linear / LTF score	なし / 0	.8220	.6502	.5690	.5682	2.28	17.0
linear / LTF score	あり / .3	.8222	.6772	.5898	.6396	2.38	17.2
LinearTurboFold	—	.8452	.6559	.5509	.6255	7.82	39.2

読み方： nonlinear は ρ=.075 、 linear は LinearTurboFold 由来 score で ρ=.3 。 linear では MCC ・ CBP F1 ・ SCI が改善し、 LinearTurboFold 本体より少ない計算資源で動く。

値は Murlet 13 cases の平均。 RIBOSUM ありの重みは、 nonlinear / linear を別々に評価した結果を採用。

16S： linear 化で長鎖の時間・メモリを抑える

RIBOSUM ありは nonlinear ρ=.075 、 linear ρ=.3 。 LinearTurboFold を同じ長鎖入力群で比較する。

共通設定

25 groups ・ --no-alifold ・ --dynamic-cbp ・ --max-iter=20
OMP_NUM_THREADS=1 ・ 入力長 1440–1560 nt

LINEAR / RIBOSUM ON

ρ=.3

LinearAlign ［LTF score］ + lpc

構成	RIBOSUM P	SCI	WALL S	RSS MIB	比較
nonlinear	なし / 0	.9724	67.27	217.8	原来経路
nonlinear	あり / .075	.9739	68.04	218.6	RIBOSUM on
linear / LTF score	なし / 0	.9489	37.12	176.1	linear path
linear / LTF score	あり / .3	.9555	38.65	177.7	RIBOSUM on
LinearTurboFold	—	.9398	43.95	352.5	external baseline

要点： RIBOSUM あり同士で linear は nonlinear より約 1.76 倍高速、 RSS 約 19% 削減。 LinearTurboFold より約 1.14 倍高速で、 RSS は約 50% 少ない。

SCI は共通構造の保全指標。 SPS / MCC / CBP F1 は長鎖ベンチマークでは未集計。

Stored analysis: results-comparison/dafs-16s, results-linear-ribosum-03/16s, results-comparison/linearturbofold-16s

23S： さらに長い入力で linear 化の差が広がる

23S でも RIBOSUM ありの方式別条件を固定し、 nonlinear ・ linear ・ LinearTurboFold を同じ表で比較する。

共通設定

```
5 groups  ·  --no-alifold  ·  --dynamic-cbp  ·  --max-iter=20
OMP_NUM_THREADS=1  ·  入力長 2850–2926 nt
```

LINEAR / RIBOSUM ON

ρ=.3

LinearAlign ［LTF score］ + lpc

構成	RIBOSUM P	SCI	WALL S	RSS MIB	比較
nonlinear	なし / 0	.8736	479.25	813.0	原来経路
nonlinear	あり / .075	.8776	485.82	812.8	RIBOSUM on
linear / LTF score	なし / 0	.8519	103.01	427.1	linear path
linear / LTF score	あり / .3	.8627	101.64	429.8	RIBOSUM on
LinearTurboFold	—	.8508	131.51	791.6	external baseline

要点： RIBOSUM あり同士で linear は nonlinear より約 4.78 倍高速、 RSS 約 47% 削減。 LinearTurboFold より約 1.29 倍高速で、 RSS は約 46% 少ない。

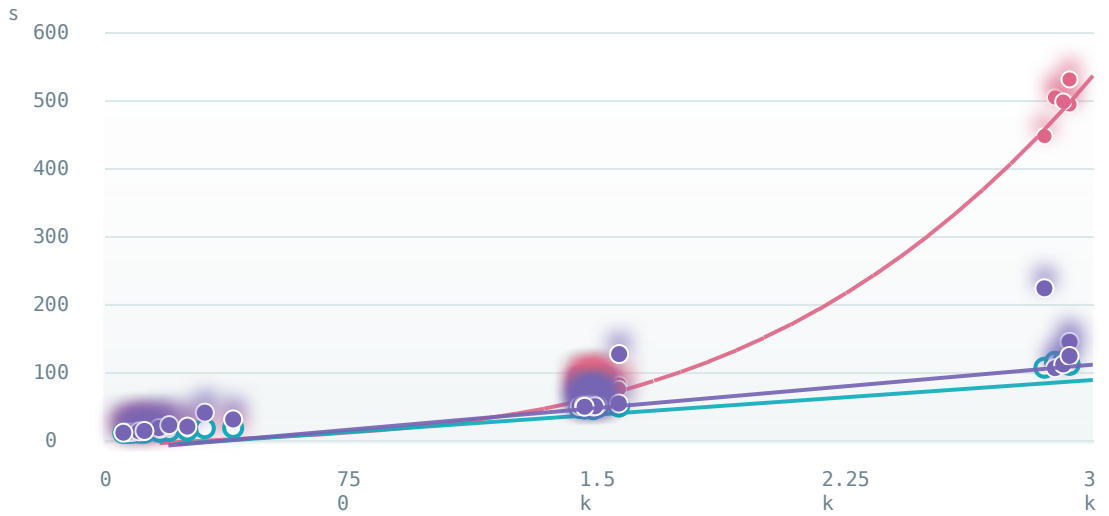
23S は速度・メモリ差が最も明瞭。正しい LinearAlign 条件では SCI .8627 で、 LinearTurboFold の .8508 も上回る。

Stored analysis: results-comparison/dafs-23s, results-linear-ribosum-03/23s, results-comparison/linearturbofold-23s

RIBOSUM あり： nonlinear ・ linear ・ LinearTurboFold の長さ依存

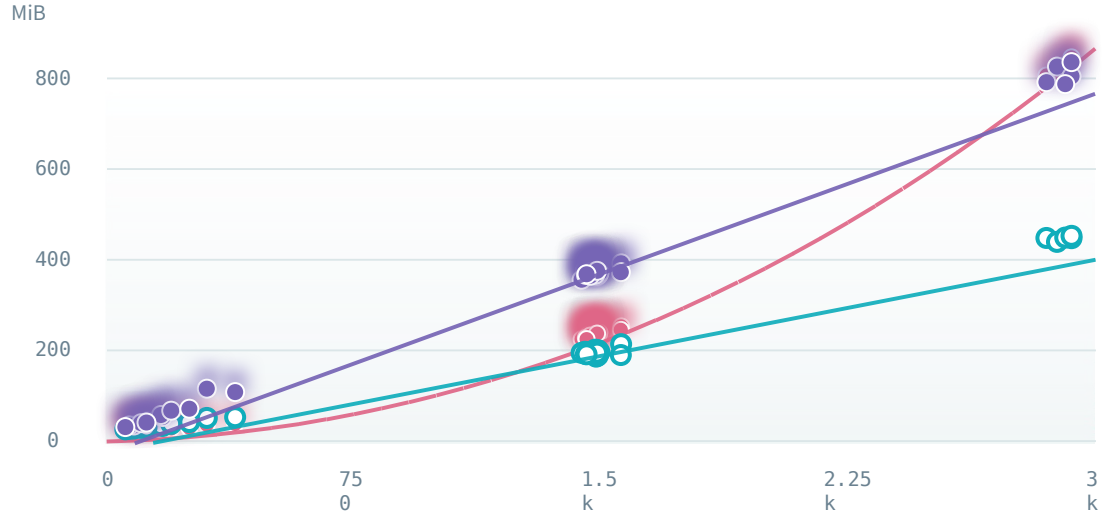
横軸は入力 FASTA の最大配列長。点は個別入力、線は方式ごとのフィットで、色は方式だけに対応する。

実行時間（秒）



nonlinear は 3 次関数、 linear / LinearTurboFold は 1 次関数でフィット。

最大メモリ（ MiB ）



RIBOSUM ありの条件を方式間で比較。ファミリー色は使わない。

nonlinear （ ρ=.075 ）　linear ［ LTF score ］（ ρ=.3 ）　LinearTurboFold　個別点　ー フィット

TIME FIT

```
nonlinear: cubic fit · R²=0.998
linear: linear fit · slope=34.0/1000 nt · R²=0.962
LTF: linear fit · slope=40.9/1000 nt · R²=0.718
```

MEMORY FIT

```
nonlinear: cubic fit · R²=1.000
linear: linear fit · slope=143.3/1000 nt · R²=0.977
LTF: linear fit · slope=265.7/1000 nt · R²=0.988
```

点数は各方式 43 点（ Murlet 13 、 16S 25 、 23S 5 ）。 nonlinear は 3 次、 linear と LinearTurboFold は 1 次の最小二乗フィット。長さ方向の傾きを方式間で比較するため、ファミリー別の色分けはしていない。

前回資料からの差分：線形化の先を評価できる段階へ

前回デッキ（9 月初旬・基準 commit 9360ad8 ）以降、プロフィールモデル、 DD 収束、条件探索を追加で検証した。

PROFILE MODEL

LinearAlifold

線形プロフィール PF を統合

Inside / outside からプロフィールの分配関数と BPP を計算。 RNAalifold に近い熱力学モードと、 progressive / final 段階の切替を追加。

DD / CERTIFICATES

85 cases

収束と停止を切り分け

上界、 Polyak 更新、実行可能解の回収を個別に評価。厳密な小規模診断と、反復予算・実行時間の比較を追加。

BENCHMARK SCALE

27,200 runs

13 例の比較から全 85 例へ

Murlet dataset1 の 320 条件を 85 ケースで再評価。さらに LinearAlifold の反復数スイープ 3,400 run で、品質と時間の変化を測定。

OPERATING POINT UPDATE

LinearAlign + LPC： RIBOSUM ρ = .3 → .5

85 ケースの MCC 重視スイープで選び直した値。前回の .3 とは評価対象と条件探索範囲が異なる。

UPDATED TAKEAWAY

精度・収束・速度を別々に評価

新機能を足すだけでなく、改善した箇所と効果が出なかった箇所を実験で分けた。

Update window: 2026-09-03 → 2026-09-21 · docs/handoff_2026-09-18_ja.md · docs/murlet_dataset1_condition_sweep_2026-09-18_ja.md

LinearAlifold を統合し、効く段階を分けて測った

線形プロフィール folding の確率計算を整え、 progressive alignment と final structure prediction の影響を分離した。

IMPLEMENTATION

プロフィールの確率を inside / outside で計算

ViennaRNA 2.7.2 の熱力学関数を使う。 PF と BPP は LinearFold beam で計算し、 DAFS の RIBOSUM は profile PF から分離する。

progressive merge → final structure

検証： 130 個の単一配列 profile で logZ/BPP を ViennaRNA と照合。 CTest 17/17 成功。 multi-sequence profile 全体が RNAalifold と同一になる主張ではない。

切替： `--alifold-stages` で段階を選び、 `--linear-profile-energy` で熱力学モードを比較可能。

STAGE ABLATION / LPC × LINEARALIGN / 85 CASES

有効段階	MCC	SCI	SPS	秒 / 例
なし	.6881	.6721	.7569	2.18
progressive	.6909	.6966	.7456	4.42
final	.6867	.6721	.7569	2.38
両方	.6913	.6966	.7456	4.57

max-iter=100 。 progressive は SCI を上げる一方、 SPS を下げ、約 2.2 秒 / 例を追加。 final 単独ではアラインメント指標は変わらず、 MCC への効果も小さく、組合せ依存。

今回の結論： LinearAlifold の追加を一律の精度改善とは扱わない。適用段階で構造精度・ alignment 保全・計算時間のトレードオフが変わる。

85 ケースで条件を再探索し、反復数の効果も測った

小規模な Murlet 比較から全 dataset1 へ広げ、 RIBOSUM 重みと DD 予算を同じ参照指標で評価した。

WEIGHT RE-SELECTION

ρ = .5

LinearAlign + LPC

85 ケース、 320 条件、 27,200 run の探索で、 MCC 最良は RIBOSUM .5 、 final weight .01 、 align-th .02 。

MCC .6851 · SCI .6638 · SPS .7572

前回の ρ=.3 は 13 例の比較で使った条件。今回の値は拡張データ上で選び直した値で、独立 held-out test の結果ではない。

LINEARALIFOLD MODE / LPC × LINEARALIGN / 85 CASES

MAX-ITER	MCC	SCI	SPS	秒 / 例
100	.6884	.6990	.7449	5.79
500	.6907	.7578	.7431	8.26
1000	.6942	.7764	.7440	11.36

beam 幅 100 、 LinearAlifold + RNAalifold 互換 profile energy を固定。 100→1000 反復で SCI/MCC は上昇したが、 wall time も 5.79→11.36 秒 / 例へ増加。

読み方： 最良条件はデータ範囲、指標の優先順位、 folding/alignment の組合せ、 DD 反復数に依存する。ここでは Murlet dataset1 上の探索結果として示す。

DD の改善は小幅、デコーダー最適化は出力を保って短縮

収束を速める案を複数比較した結果、 certificate tightening の停止効果と、 kernel-level の実行時間短縮を区別できた。

CERTIFIED STOP

上界を締めると反復を節約

1.7–6.0%

max-iter=500 で平均 DD 反復を削減。 gap は約 0.66–1.30% へ縮小。

85 例 ×4 条件、 1,020 予測が対照とバイト一致。大幅な全体高速化ではなく、証明できて停止する merge が増えた。

DD SEARCH

反復短縮の一般解は未達

100 ≠ 500

全 85 例で 100 反復は 500 反復と同等でない（ ΔMCC −.0052 、 ΔSCI −.0562 ）。

projected-norm と column-bound は全体加速を示さず、実験用・既定 OFF 。回収頻度の変更も一律の改善には至らなかった。

DECODER KERNEL

LinearFold の計算を再利用

−6.52%

平均 wall 2.434 → 2.276 秒 / 例（ Murlet 85 例 ×4 条件）。

340 run で予測・品質指標・非時間 trace が完全一致。 BPP/logZ の直接 probe も bitwise 一致。

現在のまとめ： 証明を保った一部の DD 停止短縮と、予測を変えない約 6.5% のデコーダー高速化を確認。一方、 DD 反復数を大きく減らす一般策は今後の課題。

FULL CONDITION SWEEP · NONLINEAR

非線形 fold × align： Alifold 有無ごとの最良条件

85 ケースで条件を探索。スコアは MCC → SCI → SPS の辞書式順で選択。

FOLD × ALIGN	CONSENSUS FOLDING	BEST CONDITION	DD / FINAL P	MCC	SCI	SPS	秒 / 例
CONTRAfold × CONTRAlign	RNAalifold on	`cc_alifold_fth03`	.10 / 0	**.6997**	.6916	.7707	1.73
CONTRAfold × CONTRAlign	off	`cc_noalifold_w2`	.10 / 0	**.6877**	.6897	.7726	.97
ViennaRNA × ProbCons	RNAalifold on	`vp_alifold_w2`	.02 / 0	**.6820**	.6762	.7654	1.61
ViennaRNA × ProbCons	off	`vp_noalifold_dd_0p15`	.15 / 0	**.6431**	.6835	.7633	.84

CONTRAfold × CONTRAlign: Alifold ありの best MCC は .6997 。 off 条件より +0.0120 。

ViennaRNA × ProbCons: Alifold ありで MCC +0.0389 。 SCI は off の方が高く、目的指標の選択が重要。

ρ は DD 反復／final 段階の RIBOSUM weight 。選択条件内には fold-th や weight なども含む。 Alifold on/off は各々の探索最良条件どうしの比較で、単一要因だけの因果効果ではない。

線形 fold × align： LinearAlifold 有無ごとの最良条件

folding 2 系統 × alignment 2 系統を個別に探索。 Alifold を入れれば常に MCC が上がる、とは限らない。

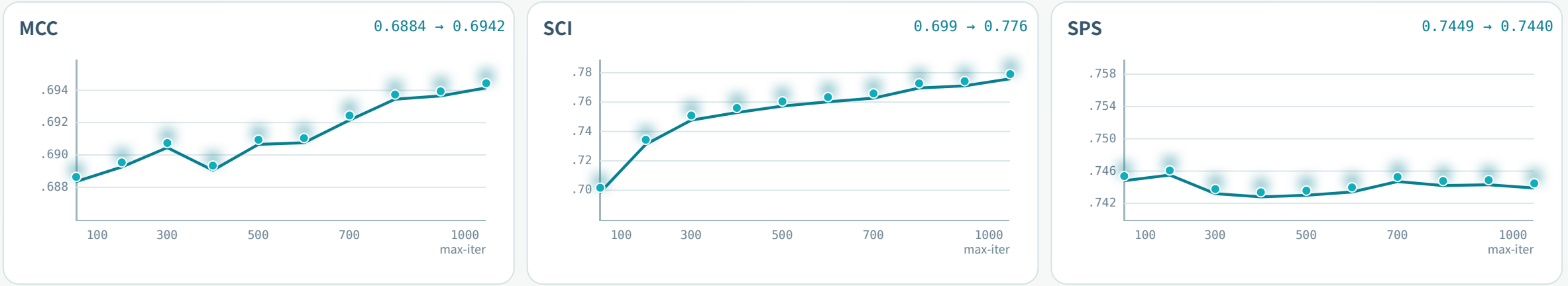
FOLD × ALIGN	LINEARALIFOLD	BEST CONDITION	DD P / KEY SETTING	MCC	SCI	SPS	秒 / 例
lpc × LinearAlign	on	`cl_linear_dd_0_alifold`	0 / align-th .005	**.6926**	.7000	.7787	5.71
lpc × LinearAlign	off	`cl_linear_ath02`	.50 / align-th .02	**.6881**	.6721	.7569	2.47
lpc × LinearCONTRAlign	on	`lpc_contralign_w2_alifold`	.30 / weight 2	**.6927**	.6749	.7550	5.90
lpc × LinearCONTRAlign	off	`lpc_contralign_w8`	.30 / weight 8	**.6864**	.6788	.7497	2.37
lpv × LinearAlign	on	`lpv_linear_dd_0p01_alifold`	.01 / DD phase	**.6809**	.6982	.7769	5.78
lpv × LinearAlign	off	`lpv_linear_fpct05`	.30 / align-pct .5	**.6694**	.6809	.7728	2.65
lpv × LinearProbConsRNA	on	`lpv_probcons_w2_alifold`	.30 / weight 2	**.6860**	.6789	.7536	5.85
lpv × LinearProbConsRNA	off	`lpv_probcons_dd_0p075`	.075 / weight 4	**.6648**	.6760	.7671	2.32

ρ は DD 段階の RIBOSUM weight 。各条件の探索範囲で final weight や align/fold 閾値も調整。 LinearAlifold on では SCI が上がる条件が多い一方、 SPS と計算時間にトレードオフがある。

読み分け： この表は Murlet dataset1 全 85 例・ 40,800 run の組合せ別探索。以前の 13 例比較で選んだ LinearAlign ρ=.3 は別の探索スコープであり、 LinearCONTRAlign の条件と取り違えない。

反復数を増やすと品質・制約違反・時間はどう動くか

lpc × LinearAlign 、 LinearAlifold + RNAalifold 互換 profile energy （ ρ=.5 ）を固定し、 max-iter だけ変更。



100 → 1000: MCC .6884 → .6942 、 SCI .6990 → .7764 、 SPS はほぼ横ばい（ .7449 → .7440 ）。違反 /merge は 28.56 → 10.49 、実行時間は 5.79 → 11.36 秒 / 例。

85 ケースの macro average 。違反 /merge ＝各 merge 最終反復の dd_summary.violated 平均。グラフはこの固定条件内の推移で、前スライドの全条件探索値とは異なる実験世代・目的のため直接差分比較しない。

線形 folding 用の LinearIPknot decoder も実装

疎な塩基対事後確率から、交差塩基対を複数レイヤーへ分けた構造を固定 beam で構成する。

01

疎な pair posterior

LinearPartition の候補支持とラグランジュ補助スコアを入力。

›

02

しきい値ごとに layer 化

各 level で正の候補だけを走査し、すでに使った塩基を除く。

›

03

LinearNussinov

各 level を固定 beam の線形 DP で非交差 matching として decode 。

›

04

交差を別 level へ

次 level には既選択 pair と交差する pair を許し、括弧 layer を出力。

DAFS への接続と検証範囲

- linear folding + `--ipknot` では `LinearIPknot` を選択。非線形側は従来 IPknot 。
- dense/sparse decoder interface 、 sparse structure storage 、構造 bounds を接続。
- unit / pipeline tests で交差 layer 、括弧整合性、 sparse/dense 一致、 decoder 選択を確認。

位置づけ

これは IPknot の厳密な整数計画を線形時間で解くものではなく、固定 beam の layer-wise 近似 decoder 。ビーム幅・しきい値・候補支持に依存する。

このスライドで示すのは実装原理とテスト範囲。既存の厳密 IPknot に対する精度・速度の定量比較は、まだ提示できるベンチマーク結果がない。

src/ipknot.h · src/ipknot.cpp · src/dafs.cpp · tests/linear_ipknot_test.cpp · tests/linear_ipknot_pipeline.cmake · tests/ipknot_sparse_support_test.cpp